



**Embedded Systems
Electronics, Power and Automation**

C style and programming guidelines

Jos Onokiewicz

6 July 2016
Version 3.1

Document history

Date	Version	Who	Changes
24-08-2015	3.0	Jos Onokiewicz	Hungarian notation removed. Indent convention adjusted. Layout in accordance with the book “In zee met C” as much as possible.
10-05-2016	3.0	Jos Onokiewicz	Layout { } instructions supplemented.
06-07-2016	3.1	Jos Onokiewicz	C11 one line comments allowed.

HAN University of Applied Sciences

**Embedded Systems
Electronics, Power and Automation**

CONTENTS

Preface.....	5
1. C standards and guidelines	7
1.1 C standards.....	7
1.2 Objective of layout guidelines	7
1.3 Objective of programming guidelines.....	7
1.4 Supplemental professional guidelines and tooling.....	8
1.5 Required and recommended guidelines.....	8
2. General guidelines	9
2.1 Make arrangements in unexpected situations.....	9
2.2 Applying guidelines to external pre-existing code.....	9
2.3 Coding in English	9
2.4 Use a code formatter for the layouts of programs	9
2.5 Do not apply guidelines afterwards in a development path.....	10
2.6 Deliver an end product of good/maximum quality.....	10
3. File naming conventions	11
3.1 Use the file name extensions .h and .c.....	11
3.2 Use informative names for files.....	11
3.3 Avoid spaces in names for directories and files	11
4. Naming convention for variables, constants and functions	12
4.1 Use lower case for keywords in C	12
4.2 Use an informative naming convention for the names of variables	12
4.3 Use lower camel case notation for the names of variables.....	12
4.4 Use p as a prefix in the names for pointers	13
4.5 Use the postfix _t for names in typedefs	13
4.6 Use upper case letters for names of constants.....	13
4.7 Use lower camel case notation for the names of functions.....	14
5. Code layout and structure of code	15
5.1 For indentation, use 3 or 4 spaces, and avoid TABs.....	15
5.2 Place accolades of code blocks consistently.....	15
5.3 Use extra spaces for better legibility	16
5.4 Avoid unnecessary empty lines	17
5.5 Use a maximum of 1 statement per line	17
5.6 Layout and code structure of a small non-modular program	17
5.7 Layout and code structure of modular programs	20
5.8 Layout of structured data types: enum, struct and union.....	20
5.8.1 enum.....	20
5.8.2 struct.....	20
5.8.3 union.....	21
5.9 Layout of statements: if-else, switch, while, do-while and for	21
5.9.1 if-else.....	21
5.9.2 switch	23
5.9.3 while	23
5.9.4 do-while.....	24
5.9.5 for.....	25
5.10 Layout parameter list of the invocation of functions	25

6.	Modular program structure	27
6.1	Divide programs into modules.....	27
6.2	Use the name of the module in the related header and implementation file	28
6.2.1	Use include guards in header files	30
6.2.2	Include the header file of a module in the implementation file as well ..	31
6.2.3	Make local data and functions in a module static	31
7.	Programming guidelines	33
7.1	Adapt code so that compiler warnings no longer appear	33
7.2	Only add useful supplementary comments that are explanatory	33
7.3	Avoid including unnecessary header files	33
7.4	Avoid C++ language elements in ANSI/ISO C programs	33
7.5	Initialise variables at the declaration	33
7.6	Use (void) for empty parameter lists of functions.....	34
7.7	Avoid "magic numbers" by using constants	34
7.8	Split functions that are too large into smaller subfunctions	35
7.9	Separate calculation and monitor functions from the user interface code ...	35
7.10	Mainly invoke functions in main()	35
7.11	Use const.....	35
7.12	Avoid global variables as much as possible	36
7.13	Do not make declarations of variables in header files.....	36
7.14	Make the interface in header files as small as possible.....	36
7.15	Avoid the use of copies of code rules	36
7.16	Test the error value that functions render	37
7.17	Avoid unnecessary dynamic memory allocation	37
7.18	After applying free(), make the pointer used equal to NULL	37
7.19	Avoid the use of signed and unsigned integers in one expression	37
	APPENDIX: Priority and associativity of operators.....	39

PREFACE

This book attempts to contribute to the improvement of the quality of C programs. The emphasis here is mainly on conventions for naming and layout. Brief consideration is also given to the structure of code and header files of C modules.

The notion that a good and consistent layout makes an important contribution to the program quality is not new. However, a good layout most certainly is not sufficient as a condition for good program quality. After all, a good designing method is also very important, where a modular approach is the priority. We indicate briefly what is meant by a modular approach for C code implementation files and related header files. How you can arrive at a good modular structure is not included in the scope of this book.

“Programs are meant to be read by humans and only incidentally for computers to execute.”

H. Abelson and G. Sussman

Some attention has been given to code formatters and the application of Doxygen for generating API documentation.

The guidelines provided in this document have a limited scope and are only intended for beginning designer trainees to become familiar with them. The guidelines can only be understood if the reader already has sufficient knowledge of the ANSI/ISO C programming language (C89, C99 or C11).

Arnhem, July 2016
Jos Onokiewicz

1. C STANDARDS AND GUIDELINES

This chapter deals with the existing standards for C and the objective and importance of layout and programming guidelines. It also indicates which tooling could be used as a supplement to the guidelines listed in this document.

1.1 C standards

The guidelines are based on ANSI-C (C89 of 1989), also referred to as ISO-C (C90 of 1990). This standard is also referred to as ANSI/ISO C. The newest standard is C11 of 2011, which has replaced the standard of 1999.

The abbreviations **ANSI** and **ISO** stand for:

- American National Standards Institute
- International Organisation for Standardisation

1.2 Objective of layout guidelines

The idea of layout guidelines is to arrive at a consistent layout of C programs. A consistent layout increases the legibility and comprehensibility and, therefore, the testability, maintainability and transmissibility. Often, more time is spent reading the code than writing the code. Therefore, the layout is one of the most important quality criteria of computer programs.

Practice (rough rule of thumb):

- 20% of the development time lies in the making of the first version of a program, 80% is spent on further maintenance.
- Generally, maintenance is not done by the developers of the first version.

Therefore, maintenance must be able to be carried out as comfortably and effectively as possible because it constitutes a large part of the realisation of software that has often been made by someone else.

There are many different (personal) opinions about layout. In this collection of guidelines, an attempt has been made to use guidelines that are frequently used in the professional domain. Sharing the same guidelines in a development team will greatly increase the comfort and efficiency of collaboration.

1.3 Objective of programming guidelines

Certain constructions in C that have been selected by a developer with not enough knowledge of C can lead to poorly working programs. When developing **safety critical** systems in the embedded system world in particular, it is essential that the correct C constructions with predictably correct behaviour be used. Programming guidelines must lead to an important contribution to this effort.

1.4 Supplemental professional guidelines and tooling

For professional embedded system development, **MISRA-C** program development guidelines from the automotive industry and the **CERT C Secure Coding Standard** are excellent starting points for an extensive supplement to the programming guidelines of this document.

Source MISRA: <http://www.misra.org.uk>

C code can also be checked with, for example, programs such as **splint** (Secure Programming Lint). The use of this tooling does require a good understanding of C in order to be able to interpret the notifications so that the correct changes can be applied to the code.

1.5 Required and recommended guidelines

Generally, in practice, each guideline has an indication whether its application is advised or required. This document does not do so (yet).

2. GENERAL GUIDELINES

Guidelines can generally be applied in very many situations. This chapter deals briefly with what to do if the guidelines do not provide a solution. It also includes guidelines that are not specific to C programming.

2.1 Make arrangements in unexpected situations

The guidelines included in this document obviously do not cover all situations. If a situation occurs for which a guideline does not make provisions, make a choice that is in keeping with the following general guidelines:

- **Avoid unnecessary complexity!**
Legibility is more important than compact code that cannot be read.
- **Consult!**
If a choice must be made, then consult other developers in the project team in order to arrive at one and the same agreement.
- **Be consistent!**
Once a specific choice has been made, the team must adhere to it consistently.

2.2 Applying guidelines to external pre-existing code

In many cases, we also have to use external pre-existing C code in projects. Sometimes this pre-existing external code needs to be adjusted. In that case, generally the guideline is to use the guidelines used by that code (as shown in the code text itself) and not apply our own guidelines.

2.3 Coding in English

Computer programs are often developed in international teams. In order to arrive at global legibility, transmissibility, maintainability and testability of programs, it is, therefore, recommended to use the **English language** for programs and documentation.

2.4 Use a code formatter for the layouts of programs

Many editors used for professional code development contain an extra functionality that automatically lays out any code entered fully or partially. It is usually possible to set the automatic indentation in code blocks. This prevents having to do a lot of (boring) manual layout work. Code formatters do have to be configured beforehand in order to comply with the guidelines set in this document.

In **QtCreator**, it is possible to choose a code layout convention through the menu bar

Tools, Options ..., C++, Code Style and, if so required, adapt it. The rules that belong to a set layout convention can be save in an xml file by using the Export option. With the Import option we can load a file that contains an agreed layout convention. This makes it easy for a project team to share a layout convention. By selecting the code text that needs a layout, we can have this text indent by default with Ctrl-i.

2.5 Do not apply guidelines afterwards in a development path

If you apply guidelines at a (much) later time in a development path, you run the risk that your development activities will not work as well. This could cause an unnecessary loss of time and poorly working code at delivery. If the code quality remains insufficient, this could endanger its transmissibility to a following project. Maintenance of poor quality code is a lengthy and often very tedious job for developers!

2.6 Deliver an end product of good/maximum quality

When finishing a project, a general requirement is to deliver to the customer a code of such quality that its further maintenance (adapting, expanding and improving) is expected to be as easy as possible. Just think of the cost aspect of further maintenance! The guidelines listed in this document form a starting point for achieving this goal. Of course, we must have documented the code sufficiently and tested its operation.

3. FILE NAMING CONVENTIONS

This chapter deals with the naming conventions of files for C programs. The compiler can tell by the file name extensions that they are related to C. We must assume the following guidelines because many compilers can compile C++ as well and because there are subtle differences between C and C++.

3.1 Use the file name extensions .h and .c

Use the extensions .h and .c for C programs. Avoid the extensions common for C++ such as .cpp and .hpp.

For guidelines for avoiding C++ language elements, see section 7.4.

3.2 Use informative names for files

Use file names that have a strong relation with the purpose of the code in the files.

If it is difficult to choose an appropriate informative file name, this could be an indication that the content of this file serves several purposes and that it must be split up. See also the guidelines for modular programming in Chapter 6.

3.3 Avoid spaces in names for directories and files

It is recommended that you avoid using spaces in directory and file names. This could cause problems for programs that need to read these directories and file names.

4. NAMING CONVENTION FOR VARIABLES, CONSTANTS AND FUNCTIONS

Arrangements for the naming convention of variables and functions are essential starting points for good quality of code, providing an important contribution to the maintainability and transmissibility of code. This chapter looks at this in detail.

4.1 Use lower case for keywords in C

Keywords in C must always be written entirely in lower case in order to prevent compile time errors. For example:

```
main
struct
if
else
do
while
int
double
char
```

C programming language is case-sensitive, which means that names you create must be written with the same combination of upper and lower case letters consistently.

4.2 Use an informative naming convention for the names of variables

In order to increase the legibility and, therefore, the comprehensibility of the code, we must choose informative names. This means that the names chosen must be long enough. Avoid as much as possible abbreviations that could be misunderstood at a later time.

Code editors often include the possibility of automatically completing names so that the use of long names for variables does not mean a lot of typing.

4.3 Use lower camel case notation for the names of variables

The names of variables start with a lower case letter. Then, per segment of the name, use an upper case letter.

```
int countedPulses = 100;
double temperatureCelsius = -10.3;
double temperatureFahrenheit = 47.0;
char escapeKey = ESC;
char errorText[100] = "file is corrupt";
```

4.4 Use p as a prefix in the names for pointers

Variables names for pointers must begin with a `p`. This allows them to be recognised easily.

```
int *pInputData = NULL;
int *pData = &data;
```

4.5 Use the postfix `_t` for names in typedefs

For existing simple types and compound types, we could create a replacement informative name with `typedef`. Behind the new name, we add `_t` so that we can immediately see that this name was created with a `typedef`. This is already a common style in standard C, such as, for example: `size_t` (unsigned int). Example:

```
typedef unsigned char byte_t;

byte_t outputBuffer[10];
byte_t *pData;
```

Detailed example structs:

```
typedef struct {
    int x;
    int y;
} point_t;

typedef struct {
    point_t pointBegin;
    point_t pointEnd;
} line_t;

point_t pointStart = {12, 5};
line_t path = {{0, 0}, {10, 20}};
```

4.6 Use upper case letters for names of constants

With `#define`, defined names for constants are written with upper case letters. If a name consists of separate words, these can be separated with the `"_"` symbol in order to increase legibility. This increases the legibility and the comprehensibility of program text.

Examples:

```
#define FALSE 0
#define TRUE (!FALSE)
#define FOREVER TRUE
```

```
#define MAXSIZE 100
#define NR_NAMES (MAXSIZE + 3)
#define MIN_TEMPERATURE_CELSIUS -1.5
#define INPUT_FILE_NAME "config.dat"
```

The example above shows that constants must each be written on a new line and can be made dependent on one another. This dependence determines the sequence of defining.

In C as well, constants can be declared with `const`. This is a qualifier for variables.

```
const int MAX = 100;
const char TAB = '\t';
const double MAX_TEMPERATURE = 200.0;
const int MAX_X = inputX < 10 ? 10 : 20;
```

Make sure that the variables that are indicated with `const` in ANSI/ISO C cannot be used for the indication of the size of static arrays. This must always be done with `#define`. During compile time, the size of static arrays must be known as a constant.

4.7 Use lower camel case notation for the names of functions

Standard C functions usually have names that consist of only lower case letters. This is not as problematic because these names are often very short. Start the names of functions you created with lower case letters. If the name is longer, we use upper case letters in the different components of the name to increase the legibility. This is called **lower camel case notation**.

Examples of some names for home-made functions (the parameter lists have been omitted):

```
int openFile(...);
int fileIsEmpty(...);
double greatestValue(...);
void initialiseLaser(...);
byte_t getInputPort(...);
```

5. CODE LAYOUT AND STRUCTURE OF CODE

A consistent layout of the code increases its legibility. This contributes to finding errors in the code more quickly and makes it easier to expand the code. Using the same layout arrangement makes collaboration in a team more effective.

For the structure of code (order of statements), in many cases, the C language is even mandatory. For other situations, supplementary arrangements are necessary. This chapter discusses this in great detail. It also examines the use of Doxygen for generating API documentation.

5.1 For indentation, use 3 or 4 spaces, and avoid TABs

For indentation, it is recommended that steps of 3 or 4 spaces are used. The number of 3 or 4 spaces was chosen because it gives a good balance between legibility (what belongs to what) on the one hand and, on the other, not jumping to the right too quickly, causing the program code to not fit well on one line.

Indentation is generally done with the TAB key. The following lines are generally automatically indented by the editor. The TAB size must be set at 3 or 4 spaces in order to comply with this guideline. This means that we can continue to use the TAB key but that spaces are placed instead. In certain cases we may want to print the code or include it in a document. Then the TAB size for the printer and/or editor must also be set at 3 or 4! In order to prevent this problem, code editors can be instructed to replace the TAB with a pre-determined number of spaces.

5.2 Place accolades of code blocks consistently

Code blocks must be placed between accolades `{ }`.

```
int main(void) {  
    int x = 10;  
    ....  
  
    if(x > 3) {  
        x + = 5;  
    }  
    else {  
        ....  
    }  
    return 0;  
}
```

In general, statements should not be placed behind the accolades of code blocks. See all examples of code in this document.

An often used alternative is the following way of placing the `{ }`.

```

int main(void)
{
    int x = 10;
    ....

    if(x > 3)
    {
        x + = 5;
    }
    else
    {
        ....
    }

    return 0;
}

```

Use one consistent style for placing the {}.

5.3 Use extra spaces for better legibility

Adding spaces greatly increases the legibility of expressions.

- Place a space on both sides of the assignment operator = and the comparison operator ==.
- Always enclose binary operators (an operator that has two operands such as + and -) by spaces.
- Use a space after each comma after a parameter in the parameter list of functions.

```

result = (10 * input + 1) / MAX;

result = calculateAverage(buffer, bufferSize);

isOkay = !ready
          && findElement(pElementTable, value,
                        &elementId);

if(counter == MAX) {
    resetItems(counter, pItemBuffer);
}

```

Do not use spaces in unary operators (operators with only one operand), such as the increment and decrement operator.

No space is placed between the name of an array and the index.

```
i++;  
--j;  
amount[i] += bonus[++j];  
x = -x;
```

No spaces are placed on either side of the @dot . or the arrow -> in the specification of a record of the field of a struct.

```
typedef struct {  
    int x;  
    int y;  
} point_t;  
  
point_t point = {2, 5};  
point_t *pPoint = &point;  
  
pPoint->x = 10;  
point.x = 30;
```

5.4 Avoid unnecessary empty lines

Do not use more than one empty line to separate sections of code from one another. Remove the excess empty lines to prevent having to scroll unnecessarily in the code text. For example, it is customary to add an empty line after the declarations of variables at the top of a block of code.

5.5 Use a maximum of 1 statement per line

Do not put more than one statement per line. For example, do not allow declarations with more of the same type of variables to be on one line or allow several function calls to be on one line.

```
.... {  
    int counter = 0;  
    int numberOfItems = 10;  
  
    counter = 12 * numberOfItems;  
    start(counter);  
    resetItems(numberOfItems);  
}
```

5.6 Layout and code structure of a small non-modular program

Small programs that use a very small number of functions (3 to 5) can be included in one .c file.

The following sequence must be in the structure:

1. Introductory comments: (short) description of the objective of the program, version number, responsible author(s) and date. Mention of a copyright and/or the description of the updates per version (change history) could possibly be included.
2. `#include <....>` (in alphabetical order if possible)
3. `#define macros ....`
4. `typedefs ....`
5. Summary of prototypes of functions used with API documentation in the comments.
6. The `main()` function.
7. The implementation of the functions that are included above `main()` as prototypes.

Example with API documentation in **Doxygen** format:

```
/**
 * @file
 * This program shows some bits and bytes manipulation.
 * @version 1.6
 * @date 12/4/2016
 */
#include <stdio.h>

#define BUFFER_SIZE 4
#define SETBIT(address, bit) ((*address) |= (1 << bit))
#define CLEARBIT(address, bit) ((*address) &= ~(1<<bit))

typedef unsigned char byte_t;

/**
 * Shows the contents of a buffer (array of bytes).
 * Printed values separated by a space.
 * @pre buffer! = NULL
 * @pre bufSize > = 0
 * @param buffer pointer to start the address of the
buffer
 * @param bufSize buffer size
 */
void showBuffer(const byte_t buffer[], int bufSize);

/**
```

```

* Increments the contents of a buffer (array of bytes).
* @pre buffer! = NULL
* @pre bufSize >= 0
* @pre 0 <= incr <= 0xff
* @param buffer pointer to start the address of the
buffer
* @param bufSize buffer size
* @param incr increment value
*/
void increment(byte_t buffer[], int bufSize,
               byte_t incr);

int main(void) {
    byte_t inputBuffer[BUFFER_SIZE] = {0x01, 0x02, 0x0a,
                                         0xff};

    printf("Start bits and bytes manipulation\n");
    printf("- Initial values:      ");
    showBuffer(inputBuffer, BUFFER_SIZE);
    puts("");

    SETBIT(inputBuffer, 2);
    SETBIT(inputBuffer + 1, 2);
    CLEARBIT(inputBuffer + 3, 0);
    printf("- Changed bits:      ");
    showBuffer(inputBuffer, BUFFER_SIZE);
    puts("");

    increment(inputBuffer, BUFFER_SIZE, 4);
    printf("- Incremented values: ");
    showBuffer(inputBuffer, BUFFER_SIZE);
    puts("");
    printf("End bits and bytes manipulation\n");

    return 0;
}

void showBuffer(const byte_t buffer[], int bufSize) {
    int i;
    for(i = 0; i < bufSize; i++) {
        printf("%02x ", buffer[i]);
    }
}

void increment(byte_t buffer[], int bufSize,
               byte_t incr) {
    int i;
    for(i = 0; i < bufSize; i++) {
        buffer[i] += incr;
    }
}

```

5.7 Layout and code structure of modular programs

If we are going to program modularly, we must use implementation (.c) files and related header (.h) files. See Chapter 6 with guidelines and code examples.

5.8 Layout of structured data types: enum, struct and union

5.8.1 enum

The keyword `enum` is used for summaries. There can be more than one enumeration on a line. These are separated by a comma and a space. The content of an **enum** must be written in upper case letters; these can be compared to constants that are created with `#define`.

Example enum without typedef:

```
enum monthName {
    JAN = 1, FEB, MAR, APR, MAY, JUN,
    JUL, AUG, SEP, OCT, NOV, DEC
};

enum monthName month = MAR;
```

Example enum with typedef (use suffix `_e`). It is preferred that typedef be used.

```
typedef enum {
    JAN = 1, FEB, MAR, APR, MAY, JUN,
    JUL, AUG, SEP, OCT, NOV, DEC
} monthName_e;

monthName_e month = AUG;
```

5.8.2 struct

In general, use a typedef to give a clearly informative name to the content of a struct. Data that displays a strong coherence is placed in a struct. The content of a struct can now be transferred to a function with one parameter. This parameter is preferably a pointer to the struct that is to be transferred in order to ensure a transfer that is time efficient.

Example 1 with typedef:

```
typedef struct {
    int memberNr;
    char *pName;
    char *pAddress;
} memberData_t;
```

```
memberData_t memberList[MAXNR_MEMBERS];
```

Example 2 without typedef (with typedef is preferred):

```
struct memberData {
    int memberNr;
    char *pName;
    char *pAddress;
};

struct memberData memberList[MAXNR_MEMBERS];
```

Example 3 with pointer in the struct to a following struct (linked list):

```
typedef struct Node {
    struct Node *pNext;
    char* pName;
    char* pAddress;
} node_t;

node_t *pFirst = NULL;
```

5.8.3 union

The same applies here as to the struct declarations, adapted where necessary.
Example union (use suffix `_u`):

```
typedef union {
    byte_t byteData[20];
    int intData[5];
} data_u;
```

5.9 Layout of statements: if-else, switch, while, do-while and for

The layout of statements assumes the following guidelines: each statement begins on a new line (see 5.5), three spaces for indentation (see 5.1) and use accolades (see 5.2).

5.9.1 if-else

Examples:

```
if(number < MAX_NUMBER) {
    number++;
}
else {
```

```

    endOfTable = TRUE;
}

if((number >= MIN_NUMBER)
    && (number < MAX_NUMBER)) {
    number++;
}
else {
    endOfTable = TRUE;
}

if(number < MAX_NUMBER) {
    number++;
}
else {
    if(number == MAX_NUMBER) {
        endOfTable = TRUE;
    }
    else {
        indexError = TRUE;
    }
}

if((number >= MIN_NUMBER)
    && (number < MAX_NUMBER)) {
    number++;
    endOfTable = (number == MAX_NUMBER);
}
else {
    endOfTable = (number >= MAX_NUMBER);
    indexError = (number != MAX_NUMBER);
}

```

Example if we do not place {} behind else:

```

if((number < MIN_NUMBER) && !indexError) {
    indexError = TRUE;
}
else if((number > MAX_NUMBER) && !indexError) {
    endOfTable = TRUE;
    indexError = TRUE;
}
else if((number == MAX_NUMBER)
    && !indexError) {
    endOfTable = TRUE;
}
else {
    number++;
}

```

5.9.2 switch

Pay attention and determine whether the keyword `break` is necessary at the end of each case code! If the `break` is purposely omitted, indicate so with a comment: `/*`

`FALLTHROUGH */`

Examples:

```
switch(month) {
    case JAN:
        printf("Jan");
        break;
    case FEB:
        printf("Feb");
        break;
    ...
    case DEC:
        printf("Dec");
        break;
    default:
        errorHandler(ERR_NR_MONTH);
}
```

In the previous example, we see a default section that indicates that an error condition has occurred. This is a form of defensive programming that reveals an incorrect value of `monthName_e` that lies outside of the range of `JAN ... DEC` during runtime.

In C, an enum value is an integer type, which could lead to incorrect calculations and attributions of integer values that lie outside of the range `JAN ... DEC`.

```
switch(evError) {
    case ERR_WRITE:
    case ERR_READ:
        fprintf(stderr, "Access error\n");
        break;
    case ERR_OPEN:
        fprintf(stderr, "File not opened\n");
        /* FALLTHROUGH */
    default:
        fprintf(stderr, "Operation halted\n");
}
```

5.9.3 while

Always use opening and closing accolades, even if there is only one statement or none at all under the `while`!

Examples:

```
while(inputIsLow(INPUT02)) {
    /* empty statement */
}

while(total < budget) {
    total += amount;
}

while(total < budget) {
    total += amount;
    nrItems++;
}

while((total < budget)
      && (total < MAX_AMOUNT)
      && (nrItems < MAXNR_ITEMS)) {
    total += amount;
}

while((total < budget)
      && (total < MAX_AMOUNT)
      && (nrItems < MAXNR_ITEMS)) {
    total += amount;
    nrItems++;
}
```

5.9.4 do-while

The key word `do` is on a separate line.

Examples:

```
i = 0;
do {
    /* empty statement */
} while(i < TABLE_SIZE && table[i++] != value);

nrItems = 0;
do {
    total += amount;
    nrItems++;
} while(total < budget);

nrItems = 0;

do {
    total += amount;
    nrItems++;
}
```



```

    } while((total < budget)
           && (total < MAX_AMOUNT)
           && (nrItems < MAXNR_ITEMS));

```

5.9.5 for

Always use opening and closing accolades (compound statement), even if there is only one statement or none at all under the `for`!

Examples:

```

for(i = 0; i < MAXNR_AMOUNTS; i++) {
    amount[i] + = bonus;
}

for(i = 0; i < MAXNR_AMOUNTS; i++) {
    amount[i] + = bonus;
    totalAmount += amount[i];
}

for(i = FIRST_AMOUNT;
    i < min(LAST_AMOUNT, MAXNR_AMOUNTS);
    i++) {
    amount[i] + = bonus;
}

for(i = FIRST_AMOUNT;
    i < min(LAST_AMOUNT, MAXNR_AMOUNTS)
    && totalAmount < MAX_AMOUNT;
    i++) {
    amount[i] + = bonus;
    totalAmount += amount[i];
}

```

5.10 Layout parameter list of the invocation of functions

For the invocation of functions, the layout of parameter lists is particularly important. Note: no space after the opening bracket! Moreover, the parameters are separated from one another by a comma followed by a space.

For very long parameter lists, it is recommended that the parameters be placed aligned under each other, just like the declaration of function headers. The extra lines begin 1 position further than the opening bracket (of the parameter list.

Examples:

```

beep();

```

```
isOkay = findElement(table, value, &elementId);  
  
isOkay = findElement(table,  
                        value,  
                        &elementId);
```

6. MODULAR PROGRAM STRUCTURE

Building programs modularly is one of the most important software engineering activities of a developer. This also allows the development activities to be divided among different developers in a project team. This chapter points out the guidelines for modular programming and looks at the use of Doxygen for generating API documentation.

6.1 Divide programs into modules

To manage the complexity of a larger program, it should be built up of separate modules that we make in different C program files. This makes it possible to be able to understand, maintain and test programs better. In a project team we can also divide development tasks over the different modules as long as we have defined the interfaces properly.

The content of a module generally serves one primary purpose. A module is characterised by:

- **Logical cohesion:** the elements within a module belong together logically.
- **Cohesion:** there is a relatively strong connection between the elements (constants, types, variables, functions, etc.) within a module. They serve one primary purpose.
- **Narrow interface to other modules:** there is a relatively weak connection between the elements within a module and the outside world (hide as much as possible of the implementation details of a module).
- **Reusability:** a good module is built in such a way that reuse in other programs is easily possible.

A good modular structure is achieved if the module is a combination of data and functions that operate on that data. The functions hide the data (**data hiding**): users of the module cannot influence the data other than by means of these functions in the interface. By checking this in these functions, it can be guaranteed that the data is always consistent.

It is strongly recommended to place data with a strong cohesion in a struct. This structuring forms a step up to an object-oriented approach as is possible in C++.

NOTE: Data can also be related in embedded systems with registers in hardware.

6.2 Use the name of the module in the related header and implementation file

The name of a module can be found in the name of the header and implementation file.

For example, for the module IOcontrol:

```
IOcontrol.h
IOcontrol.c
```

Also include the header file IOcontrol.h in the implementation file IOcontrol.c (see guideline 6.2.2).

The following example is a primary program that uses the **vector2D** module. API documentation is added in **Doxygen** format.

```
/**
 * @file
 * This program shows some vector calculations:
 * - #add
 * - #distanceEndpoints
 * - #isPerpendicular
 * @author Joe Gynius
 * @version v1.5
 * @date 10/3/2016
 */

#include "vector2D.h"
#include <stdio.h>

#define NUMBER_OF_VECTORS 3

int main(void) {
    vector2D_t v1 = {0, 0};
    vector2D_t v2 = {0, 0};
    vector2D_t vectors[NUMBER_OF_VECTORS];

    v1.x = 10;
    v1.y = 20;
    v2.x = 20;
    v2.y = -10;

    vectors[0] = v1;
    vectors[1] = v2;
    vectors[2] = NULL_VECTOR;

    add(&vectors[0], &vectors[1], &vectors[2]);
```

```

printf("{%lf,%lf} + {%lf,%lf} = {%lf, %lf}\n",
        v1.x, v1.y, v2.x, v2.y,
        vectors[2].x, vectors[2].y);
printf("Distance endpoints v1 and v2 = %lf\n",
        distanceEndpoints(&v1, &v2));

if(isPerpendicular(&v1, &v2)) {
    printf("v1 and v2 are perpendicular.\n");
}
else {
    printf("v1 and v2 are not perpendicular.\n");
}
if(isPerpendicular(&UNIT_VECTOR_X, &UNIT_VECTOR_Y)) {
    printf("Unit vectors X and Y are perpendicular to "
           "each other.\n");
}

return 0;
}

```

Example interface header file of the vector2D module:

```

#ifndef VECTOR2D_H
#define VECTOR2D_H

/**
 * The struct type vector2D_t represents a 2D vector {x, y}.
 */
typedef struct {
    double x;
    double y;
} vector2D_t;

/** Null vector {0, 0} */
extern const vector2D_t NULL_VECTOR;
/** Unit X vector {1, 0} */
extern const vector2D_t UNIT_VECTOR_X;
/** Unit Y vector {0, 1} */
extern const vector2D_t UNIT_VECTOR_Y;

/**
 * Calculates the distance of the endpoints of
 * two vectors.
 * @param pv1 pointer to first vector.
 * @param pv2 pointer to second vector.
 * @return distance of the endpoints of pv1 and pv2.
 */
double distanceEndpoints(const vector2D_t* pv1,
                        const vector2D_t* pv2);

```

```

/**
 * Adds two vectors.
 * @param[in] pv1 pointer to first vector.
 * @param[in] pv2 pointer to second vector.
 * @param[out] pvResult pointer to result vector.
 * @return distance of the endpoints of *pv1 and *pv2.
 * @todo Add more vector calculation functions:
 * subtract, rotate, ...
 */
void add(const vector2D_t* pv1, const vector2D_t* pv2,
        vector2D_t* pvResult);

/**
 * Determines if two vectors are perpendicular to each other
 * by calculating the inner product.
 * @param[in] pv1 pointer to first vector.
 * @param[in] pv2 pointer to second vector.
 * @return 0 if not perpendicular, 1 if perpendicular.
 * @retval 0 vectors are not perpendicular
 * @retval 1 vectors are perpendicular
 */
int isPerpendicular(const vector2D_t* pv1,
                   const vector2D_t* pv2);

#endif

```

6.2.1 Use include guards in header files

The content of each header file must be protected by **#ifndef ... #endif** structure for multiple **#include**. The name of the header file is used in upper case letters. The dot must be replaced by a **_**.

For example, for the header file `IOcontrol.h` that contains an appropriate include guard construction:

```

#ifndef IOCONTROL_H
#define IOCONTROL_H
....
....
#endif

```

Example implementation file of the `vector2D` module:

```

#include <math.h>
#include "vector2D.h"

/** Used by #equalsZero */

```

```

#define EPSILON 1E-9

const vector2D_t NULL_VECTOR = {0, 0};
const vector2D_t UNIT_VECTOR_X = {1, 0};
const vector2D_t UNIT_VECTOR_Y = {0, 1};

/**
 * Calculates if double typed input equals 0.
 * @param i value to be checked.
 * @return |i| < EPSILON
 */
static int equalsZero(double i) {
    return (i < EPSILON) && (i > -EPSILON);
}

double distanceEndpoints(const vector2D_t* pv1,
                        const vector2D_t* pv2) {
    double deltaX = pv2->x - pv1->x;
    double deltaY = pv2->y - pv1->y;

    return sqrt(deltaX * deltaX + deltaY * deltaY);
}

void add(const vector2D_t* pv1, const vector2D_t* pv2,
        vector2D_t* pvResult) {
    pvResult->x = pv1->x + pv2->x;
    pvResult->y = pv1->y + pv2->y;
}

int isPerpendicular(const vector2D_t* pv1,
                    const vector2D_t* pv2) {
    double inproduct = (pv1->x * pv2->x) + (pv1->y * pv2->y);

    return equalsZero(inproduct);
}

```

6.2.2 Include the header file of a module in the implementation file as well

The include of the related header file can always be found in the implementation file of a module.

6.2.3 Make local data and functions in a module static

Make data and functions that are local in an implementation file but are not in the interface of a module `static`. The `static` qualifier restricts the scope to the end of the file in which they are included and are, therefore, not accessible through the linker so that they cannot be used in other files (modules).

If functions are too large, it is strongly recommended that they be divided into smaller functions (see 7.8). These subfunctions must be made `static` in the implementation file because they must not be able to be accessed by the outside world (other modules).

7. PROGRAMMING GUIDELINES

In addition to layout guidelines, programming guidelines are essential for making code that predictably works correctly. This chapter also examines guidelines that give directions for comments and coding activities that should be avoided. This chapter is by no means complete and should be regarded as merely providing guidance to the developer trainee. Supplementation can be sought in, for example, professional guidelines such as MISRA.

7.1 Adapt code so that compiler warnings no longer appear

In addition to error notifications, C compilers also produce warnings as long as this is set correctly. Compile programs with a high warning level. For example, for the GCC compiler use at least the options `-Wall -Wextra`. For even more supporting monitoring, use the extra options: `-ansi -pedantic`.

Warnings can indicate potential errors in the code! Adapt code so that warnings no longer appear. Aim for **cleanly compilable code**. It is possible to convert the GCC compiler warnings into errors using the option `-Werror`. This forces the developer to always solve warnings (which have now become errors).

7.2 Only add useful supplementary comments that are explanatory

Comments are meant to inform developers about matters that cannot be read in the code directly. By using informative names for variables and functions, comments are not usually necessary. A useful comment is, for example, an explanation as to why certain decisions were taken.

7.3 Avoid including unnecessary header files

Restrict including header files in a file to only those header files that are strictly necessary.

7.4 Avoid C++ language elements in ANSI/ISO C programs

Avoid C++ language elements, such as:

- State variables only at the beginning of code blocks.
- Use an `int` for a Boolean variable, therefore, no `bool` type.

7.5 Initialise variables at the declaration

Local variables in a code block are given a random content after declaration. If we

forget to initialise these variables (give them a start value) and use them anyway, it may cause erroneous behaviour in a program that is difficult to understand.

```
.... {
    int data = 0;
    long int code = 0L;
    int buffer[MAX_DATA] = { 1, -2, 5, -10};
    int matrix[2][2] = {{-1, 0}, {0, 1}};
    int *pBuffer = NULL;
    char text[MAX_SIZE] = "";
    ....
}
```

7.6 Use (void) for empty parameter lists of functions

Avoid the () notation for empty parameter lists of C functions, since this is the C++ notation for empty parameter lists. In the ANSI/ISO C programming language, however, this means that the compiler/linker can expect a random number of parameters! This means that a random number of parameters may also be included when invoking a function such as this one even if they are not used in the function. This indicates, in general, the invocation of a possibly wrong function.

```
/* C function prototypes */
void intialise1(void);
void intialise2();

/* C function calls */
intialise1();
intialise1(x, y); /* link time error */

intialise2();
intialise2(x, y); /* no compile/link time error!! */
```

7.7 Avoid "magic numbers" by using constants

All numbers not equal to -1, 0 and 1 are referred to as "magic numbers." A developer cannot instantly see the purpose of these numbers simply by looking at their values. It is also necessary to adjust these numbers in many places where updates are done. With this, there is the danger that numbers with the same value need not be adjusted at the same time. This gives rise to extra errors in the code that are difficult to solve.

By working with `#define` and informative names, this problem can be solved.

```
#define MAX_LINES 55
```

7.8 Split functions that are too large into smaller subfunctions

If a function consists of a large number of lines, it is more difficult to understand and therefore, maintain and test. Do not allow a function to be larger than 30 to 35 lines. Split functions that are too large into smaller subfunctions with a clear and informative name.

Make subfunctions in an implementation file of a module `static`. The prototypes of these static functions should not be included in the header file.

7.9 Separate calculation and monitor functions from the user interface code

If there is interaction by the user with the program for the input and output of data (for example, with `scanf()` and `printf()`), we must not execute this interaction in the calculation and monitor code. Use the parameter lists of the calculation and monitor functions to transfer input and output values in the interaction functions that invoke these calculation and monitor functions. This makes invoking these calculation and monitor functions in other parts of the program or even other programs possible (aiming at reuse).

7.10 Mainly invoke functions in `main()`

In general, the `main()` function should consist of invoking other functions. The advantage of this is that we can better understand `main()` because names of the invoke function are informative as to what they do. This makes the main program easier to maintain and test. Guideline 7.8 is also applicable to `main()`.

7.11 Use `const`

If we transfer data through a pointer to a function and the data indicated by the pointer may not be changed by the function, use `const`. Because the compiler now knows that the value of a `const` variable may not be changed, it gives an error notification if it is included in the code.

```
void printMembers(const member_t data[], int size);  
void printMember(const member_t *pData);
```

It is strongly recommended that `const` be applied immediately when taking the first step in the implementation of a new function.

7.12 Avoid global variables as much as possible

Avoid the use of global variables as much as possible. Make auxiliary variables local and exchange information with the outside world only by means of parameters of functions.

The only exception is the module data in an implementation file, which is global for all functions of the module; but this data must be inaccessible outside of the module (**data hiding**). This means that it must not be included in the related header file and that the access qualifier `static` must be included in the implementation file.

```
static int count = 0;
```

7.13 Do not make declarations of variables in header files

Declarations of variables must only be used in implementation files (.c files). Global variables must be stated one time in one implementation file and must be referred to with `extern` in a related interface header file:

```
extern int global;
```

If other files wish to make use of this global data, the related header file must be included in these files with `#include`.

7.14 Make the interface in header files as small as possible

Reduce the number of interface components in a header file to a minimum:

- Do not include any prototypes of the subfunctions in a header file; these must be included as `static` in the related .c file.
- In a header file, only include the `#include` of other header files that must be used in the interface.
- Avoid including `#defines` in a header file that will only be used locally in the related .c file. Put the local `#defines` at the top of the .c file.

7.15 Avoid the use of copies of code rules

Avoid copying lines of code that belong together to other parts of a program. Use a function with fitting parameters for these code rules and evoke this function in the different places.

7.16 Test the error value that functions render

Many C functions render an error value. In general, it is customary to see a value unequal to 0 as an error condition. If the error condition appears in the function invoked, the program will display undefined behaviour after executing further steps. This must be avoided at all times.

7.17 Avoid unnecessary dynamic memory allocation

Dynamic memory allocation with `malloc()` and deallocation with `free()` should only be applied if we know how much data is to be allocated during the runtime. Dynamic memory allocation can easily result in **memory leakage** due to incorrect programming.

Extra tooling in addition to the compiler can check for memory leakage in a program by evaluating the memory leakage during execution. An example of tooling in Linux: **valgrind**.

In embedded systems with **safety critical** aspects, avoid the use of dynamic memory allocation in the code related to safety critical functionality.

7.18 After applying free(), make the pointer used equal to NULL

If, through a program error, the pointer used for dynamic memory allocation is used a second time by `free()` after the execution of `free()`, the program will crash. This can be prevented by making this pointer equal to `NULL` immediately after `free()`. Freeing the memory to which a `NULL` pointer refers has no effect and does not lead to runtime errors.

7.19 Avoid the use of signed and unsigned integers in one expression

C makes use of implicit conversions of types. This is necessary in expressions with different data types. For example, the use of `int` and `unsigned int` in one expression. This can lead to unexpected results of calculations. Problems can be prevented by explicitly casting the unsigned type to an `int`.

Example:

```
unsigned int a = 123;
unsigned int b = 0;
int c = -123;

b = a + c;
printf("%u\n", b);
```

```

if (a < c) {
    printf("123 < -567\n");
}
else {
    printf("OK\n");
}

/* explicit type casting */
if ((int)a < c) {
    printf("123 < -567\n");
}
else {
    printf("OK\n");
}

```

In a certain system, this gives the following output, from which is apparent that the results are incorrect:

```

4294966852
123 < -567
OK

```

APPENDIX: PRIORITY AND ASSOCIATIVITY OF OPERATORS

priority	associativity	operator
1	left	() [] -> .
2	right	! ~ ++ -- (type) sizeof +(unary) -(unary) *(unary) &(unary)
3	left	* / %
4	left	+ -
5	left	<< >>
6	left	< <= > >=
7	left	== !=
8	left	&
9	left	^
10	left	
11	left	&&
12	left	
13	right	?:
14	right	= += -= *= /= %= &= ^= = <<= >>=
15	left	,

No copyrighted work was included in this educational publication.